

系统开发工具基础

第四周实验报告

姓名：林子钰

学号：25060012017

班级：计算机类 1 班

2026 年 9 月 16 日

目录

1	实验概述	3
2	实验环境	3
3	课堂实验	4
3.1	实验十三：建立可执行的本地质量门禁	4
3.1.1	实验目的	4
3.1.2	项目配置与测试	4
3.1.3	质量问题修复	6
3.1.4	实验结果	8
3.1.5	问题分析与体会	10
3.2	实验十四：让 Make 只重建真正受影响的产物	10
3.2.1	实验目的	10
3.2.2	源文件与 Makefile	11
3.2.3	增量构建验证	12
3.2.4	实验结果与提交记录	13
3.2.5	问题分析与体会	14
3.3	实验十五：把本地 API 数据转换为可读报告	15
3.3.1	实验目的	15
3.3.2	启动服务并获取数据	15
3.3.3	筛选、排序与报告脚本	16
3.3.4	实验结果	18

3.3.5	问题分析与体会	19
3.4	实验十六：修复并交付一个陌生的小型工具仓库	19
3.4.1	实验目的	19
3.4.2	检查源码和测试	19
3.4.3	故障复现与修复	22
3.4.4	Makefile、构建与哈希	24
3.4.5	实验结果与体会	26
4	课后练习	27
5	课后练习一：配置格式化器、linter 与 pre-commit 钩子	27
5.1	AI Agent 运行成果（无截图，正文留档）	28
6	课后练习二：单元测试、覆盖率报告与 AI 辅助提升	29
6.1	AI Agent 运行成果（无截图，正文留档）	30
7	课后练习三：配置持续集成并在每次 push 时运行	30
8	课后练习四：使用 grep 与 semgrep 查找危险调用	33
9	课后练习五：使用正则查找替换 Markdown 项目符号	34
10	课后练习六：从 JSON 中捕获 name 并改用 JSON 解析器	35
11	Git 仓库与提交记录	37
12	实验总结	37

1 实验概述

第四周实验围绕代码质量门禁、构建系统、API 数据处理、综合仓库修复，以及课后工程化练习展开。本周课堂实验对应课程第 13 题至第 16 题，分别完成 `ruff` 与 `pytest` 质量检查、只重建受影响产物的 `Makefile`、使用 `curl` 和 `jq` 生成 Markdown 报告，以及在陌生小型仓库中完成失败复现、修复、构建、哈希校验和 `Git` 提交。

在 Q13 中，报告复用了 `greetlab` 包并建立本地质量门禁。项目同时检查格式、静态分析和自动化测试，最终由 `check.sh` 统一执行。该实验说明质量工具不能只“安装成功”，还必须被组合成可重复、可返回明确退出码的检查流程。

在 Q14 中，`Makefile` 根据源文件、脚本和生成物之间的时间戳关系决定是否重新执行配方。第一次 `make` 生成结果，第二次 `make` 不执行任何配方，修改 `data.csv` 后才重新生成中间文件与最终报告，体现了声明式构建系统对增量构建的支持。

在 Q15 中，本地 HTTP 服务把 `packages.json` 暴露为 API，`curl` 负责可靠获取数据，`jq` 负责筛选、排序和生成三列表格，最终写入 `summary.md`。该过程体现了 Unix 工具通过标准输入输出组合完成数据处理的能力。

在 Q16 中，先故意破坏问候语并确认测试能够捕获错误，再修复实现并补齐 `Makefile`、`Wheel` 构建、SHA-256 校验和 `Git` 提交。和单纯的“把代码改对”相比，本实验更强调修复后仍要重新运行完整检查，并保留可审计的构建和提交证据。

课后练习还覆盖了 `pre-commit`、单元测试与覆盖率、CI、`grep/semgrep`、正则查找替换以及 JSON 解析。对于未保存 AI agent 截图的环节，本报告直接保留提示词、执行命令、关键输出和结论，使 AI 参与过程仍可审查、可复现。

2 实验环境

表 1: 实验环境

项目	配置
操作系统	Ubuntu 24.04.4
Shell	Bash
Python 版本	Python 3.10.12
质量检查工具	Ruff、pytest 9.1.1
覆盖率工具	coverage.py 7.1.0
构建工具	GNU Make、Python build 1.2.0、setuptools 26.3
API 工具	curl、jq 1.6、Python http.server
版本控制	Git
编辑器	GNU nano
semgrep 状态	当前虚拟环境中未安装，命令返回 No such file or directory

3 课堂实验

3.1 实验十三：建立可执行的本地质量门禁

3.1.1 实验目的

1. 复用 Q10 的 greetlab 项目并加入最小质量工具配置。
2. 为正常姓名和空白姓名分别编写带有效断言的测试。
3. 使用 `ruff format`、`ruff check` 和 `pytest` 修复全部问题。
4. 编写统一的 `check.sh`，保证全部检查通过时返回 0。
5. 通过多次提交保留配置、测试和质量脚本的演进过程。

3.1.2 项目配置与测试

在 `pyproject.toml` 中加入 Ruff 与 pytest 的最小配置：

```
1 [tool.ruff]
2 line-length = 88
3
4 [tool.ruff.lint]
5 select = ["E", "F", "I"]
6
7 [tool.pytest.ini_options]
8 testpaths = ["tests"]
```

测试同时覆盖正常输入和空白输入。空白姓名应当被拒绝，并产生非零退出状态；正常姓名则应在标准输出中得到完整问候语。

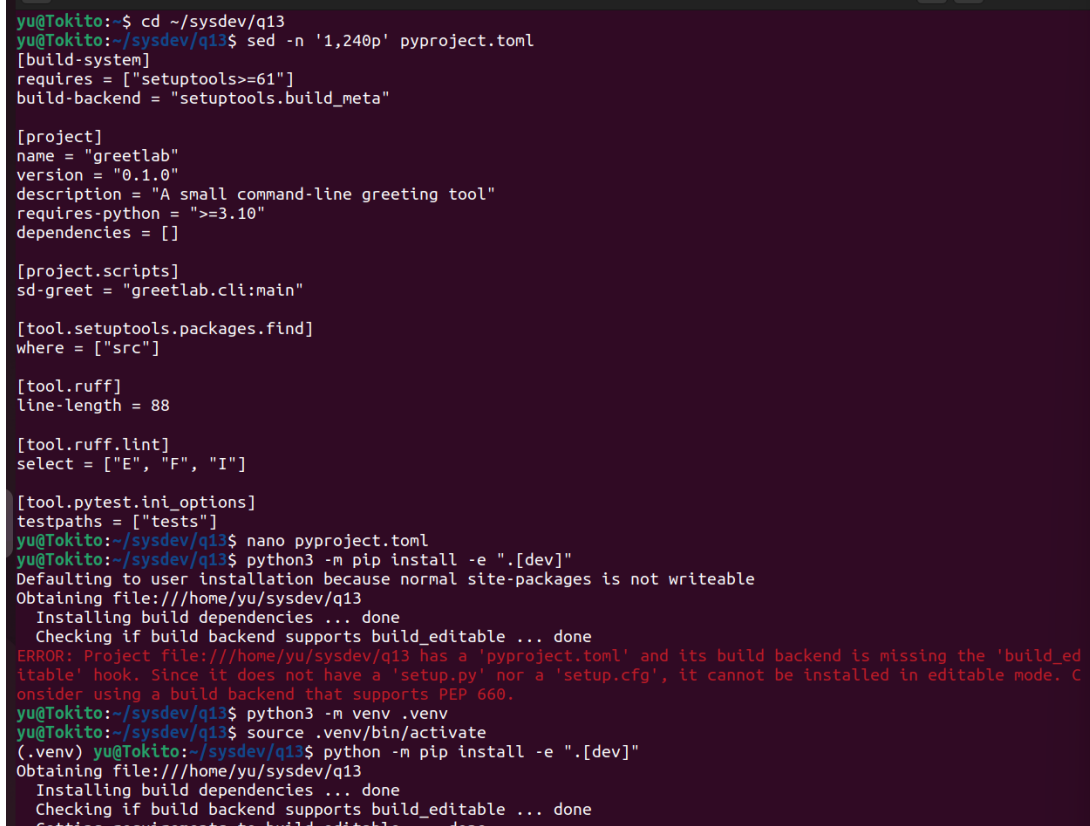
```
1 import sys
2 import pytest
3 from greetlab.cli import main
4
5
6 def test_normal_name_prints_greeting(monkeypatch, capsys):
7     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Alice"])
8     main()
9     captured = capsys.readouterr()
10    assert captured.out == "Hello, Alice!\n"
11
12
```

```
13 def test_blank_name_exits_with_code_2(monkeypatch):
14     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", " "])
15     with pytest.raises(SystemExit) as exc_info:
16         main()
17     assert exc_info.value.code == 2
```

质量门禁脚本按照题目要求的顺序执行检查：

```
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  ruff format --check
5  ruff check
6  python -m pytest
```

首次安装依赖时，项目使用 `python -m pip install -e ".[dev]"`。实验中也观察到仅有旧版 `setuptools` 时无法提供 PEP 660 可编辑安装能力，因此需要新的虚拟环境中补齐构建后端与开发依赖。配置完成后，`Ruff` 可统一格式化代码，`pytest` 正常收集两个测试。



```
yu@Tokito:~$ cd ~/sysdev/q13
yu@Tokito:~/sysdev/q13$ sed -n '1,240p' pyproject.toml
[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab"
version = "0.1.0"
description = "A small command-line greeting tool"
requires-python = ">=3.10"
dependencies = []

[project.scripts]
sd-greet = "greetlab.cli:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E", "F", "I"]

[tool.pytest.ini_options]
testpaths = ["tests"]
yu@Tokito:~/sysdev/q13$ nano pyproject.toml
yu@Tokito:~/sysdev/q13$ python3 -m pip install -e ".[dev]"
Defaulting to user installation because normal site-packages is not writeable
Obtaining file:///home/you/sysdev/q13
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
ERROR: Project file:///home/you/sysdev/q13 has a 'pyproject.toml' and its build backend is missing the 'build_editable' hook. Since it does not have a 'setup.py' nor a 'setup.cfg', it cannot be installed in editable mode. Consider using a build backend that supports PEP 660.
yu@Tokito:~/sysdev/q13$ python3 -m venv .venv
yu@Tokito:~/sysdev/q13$ source .venv/bin/activate
(.venv) yu@Tokito:~/sysdev/q13$ python -m pip install -e ".[dev]"
Obtaining file:///home/you/sysdev/q13
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
```

图 1: Q13 的 `pyproject.toml`、开发依赖安装与测试配置

3.1.3 质量问题修复

第一次运行 `ruff format` 时，Q10 复制过来的源码和测试共有 4 个文件需要重新格式化。执行 `ruff check --fix` 后，剩余的测试导入顺序问题也被修复。全过程没有在配置中全局忽略规则，而是直接修正源码与测试。

```
1 $ ruff format
2 4 files reformatted, 4 files left unchanged
3
4 $ ruff check --fix
5 All checks passed!
6
7 $ ruff format --check
8 8 files already formatted
9
10 $ python -m pytest
11 collected 2 items
12 tests/test_cli.py .. [100%]
13 2 passed in 0.08s
```

```
[ ] 4 fixable with the --fix option.
(.venv) yu@Tokito:~/sysdev/q13$ ruff format --check .
unformatted: File would be reformatted
--> q10/src/greetlab/cli.py:3:1
2 |
3 | +
4 | def main():
  |

unformatted: File would be reformatted
--> q10/tests/test_cli.py:7:1
6 |
7 | +
8 | def test_blank_name_exits_with_code_2(monkeypatch):
  |

unformatted: File would be reformatted
--> src/greetlab/cli.py:3:1
2 |
3 | +
4 | def main():
  |

unformatted: File would be reformatted
--> tests/test_cli.py:7:1
6 |
7 | +
8 | def test_blank_name_exits_with_code_2(monkeypatch):
  |
-----
15 |
16 | +
17 | def test_normal_name_prints_greeting(monkeypatch, capsys):
  |

4 files would be reformatted, 4 files already formatted
(.venv) yu@Tokito:~/sysdev/q13$ python -m pytest --version
pytest 9.1.1
(.venv) yu@Tokito:~/sysdev/q13$ python -m pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.06s =====
```

图 2: Q13 中 Ruff 格式化、pytest 运行与两测试通过结果

修复过程中还发现 `check.sh` 直接调用 `pytest` 时可能受当前 shell 的 `PATH` 影响, 最终改为显式使用 `python -m pytest`, 从而保证虚拟环境中的解释器和测试模块能够被稳定定位。

```
4 files would be reformatted, 4 files already formatted
(.venv) yu@Tokito:~/sysdev/q13$ ruff format .
4 files reformatted, 4 files left unchanged
(.venv) yu@Tokito:~/sysdev/q13$ ruff check . --fix
All checks passed!
(.venv) yu@Tokito:~/sysdev/q13$ ruff format --check .
8 files already formatted
(.venv) yu@Tokito:~/sysdev/q13$ ruff check .
All checks passed!
(.venv) yu@Tokito:~/sysdev/q13$ ^C
(.venv) yu@Tokito:~/sysdev/q13$ python -m pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.08s =====
(.venv) yu@Tokito:~/sysdev/q13$ nano check.sh
(.venv) yu@Tokito:~/sysdev/q13$ chmod +x check.sh
(.venv) yu@Tokito:~/sysdev/q13$ ./check.sh
8 files already formatted
All checks passed!
./check.sh: line 6: pytest: command not found
(.venv) yu@Tokito:~/sysdev/q13$ ruff format --check .
8 files already formatted
(.venv) yu@Tokito:~/sysdev/q13$ ruff check .
All checks passed!
(.venv) yu@Tokito:~/sysdev/q13$ python -m pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.07s =====
```

图 3: Q13 修复 Ruff 问题并验证 pytest、check.sh 退出行为

3.1.4 实验结果

最终执行 `./check.sh` 时，格式检查显示 8 个文件已经格式化，Ruff 显示所有检查通过，pytest 收集并通过 2 个测试。随后执行 `echo $?` 得到退出码 0，满足本地质量门禁要求。

```
(.venv) yu@Tokito:~/sysdev/q13$ chmod +x check.sh
(.venv) yu@Tokito:~/sysdev/q13$ ./check.sh
8 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]
===== 2 passed in 0.05s =====
(.venv) yu@Tokito:~/sysdev/q13$ echo $?
0
(.venv) yu@Tokito:~/sysdev/q13$ ruff format --check .
8 files already formatted
(.venv) yu@Tokito:~/sysdev/q13$ ruff check .
All checks passed!
(.venv) yu@Tokito:~/sysdev/q13$ python -m pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]
===== 2 passed in 0.08s =====
(.venv) yu@Tokito:~/sysdev/q13$ grep -RniE 'ignore|per-file-ignores|noqa' pyproject.toml src tests check.sh
```

图 4: Q13 最终 check.sh 通过、2 个测试通过且退出码为 0

Git 历史中可分为配置、测试和质量脚本三类提交，提交编号分别包含 07cc159、ecbf6c4 和 d2bc8d3。这种拆分方式比把全部改动压成一个模糊提交更容易审查。

```
(.venv) yu@Tokito:~/sysdev/q13$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .venv/
    ai_log.md
    check.sh
    pyproject.toml
    q10/
    src/
    tests/

nothing added to commit but untracked files present (use "git add" to track)
(.venv) yu@Tokito:~/sysdev/q13$ git add pyproject.toml
(.venv) yu@Tokito:~/sysdev/q13$ git commit -m "chore: add pytest and ruff configuration"
[master (root-commit) 07cc159] chore: add pytest and ruff configuration
 1 file changed, 31 insertions(+)
 create mode 100644 pyproject.toml
(.venv) yu@Tokito:~/sysdev/q13$ git add tests
(.venv) yu@Tokito:~/sysdev/q13$ git commit -m "test: add valid and blank name cases"
[master ecbf6c4] test: add valid and blank name cases
 2 files changed, 23 insertions(+)
 create mode 100644 tests/__pycache__/test_cli.cpython-310-pytest-9.1.1.pyc
 create mode 100644 tests/test_cli.py
(.venv) yu@Tokito:~/sysdev/q13$ git add check.sh src
(.venv) yu@Tokito:~/sysdev/q13$ git commit -m "chore: add quality check script"
[master d2bc8d3] chore: add quality check script
16 files changed, 58 insertions(+)
 create mode 100755 check.sh
 create mode 100644 src/greetlab.egg-info/PKG-INFO
 create mode 100644 src/greetlab.egg-info/SOURCES.txt
 create mode 100644 src/greetlab.egg-info/dependency_links.txt
 create mode 100644 src/greetlab.egg-info/entry_points.txt
 create mode 100644 src/greetlab.egg-info/requires.txt
 create mode 100644 src/greetlab.egg-info/top_level.txt
 create mode 100644 src/greetlab/__init__.py
 create mode 100644 src/greetlab/__pycache__/__init__.cpython-310.pyc
 create mode 100644 src/greetlab/__pycache__/cli.cpython-310.pyc
 create mode 100644 src/greetlab/cli.py
 create mode 100644 src/greetlab_25060012017.egg-info/PKG-INFO
 create mode 100644 src/greetlab_25060012017.egg-info/SOURCES.txt
 create mode 100644 src/greetlab_25060012017.egg-info/dependency_links.txt
 create mode 100644 src/greetlab_25060012017.egg-info/entry_points.txt
 create mode 100644 src/greetlab_25060012017.egg-info/top_level.txt
(.venv) yu@Tokito:~/sysdev/q13$ git log --oneline --decorate -5
d2bc8d3 (HEAD -> master) chore: add quality check script
ecbf6c4 test: add valid and blank name cases
07cc159 chore: add pytest and ruff configuration
(.venv) yu@Tokito:~/sysdev/q13$
```

图 5: Q13 分阶段 Git 提交与提交历史

3.1.5 问题分析与体会

本实验最值得注意的问题是“单独运行成功”不等于“门禁脚本成功”。工具是否位于当前解释器、脚本是否设置了 `set -euo pipefail`、工作目录是否一致，都会影响最终退出码。使用 `python -m pytest` 和 `python -m ruff` 比依赖 `PATH` 中的裸命令更稳定。质量门禁的价值在于把多个手工判断转换为一条可重复执行的命令，使 CI 和本地开发能够使用同一套标准。

3.2 实验十四：让 Make 只重建真正受影响的产物

3.2.1 实验目的

1. 理解 Make 的目标、依赖和配方三个基本组成部分。
2. 根据文件的实际依赖关系描述增量构建规则。
3. 验证无改动时 Make 不重复执行配方。

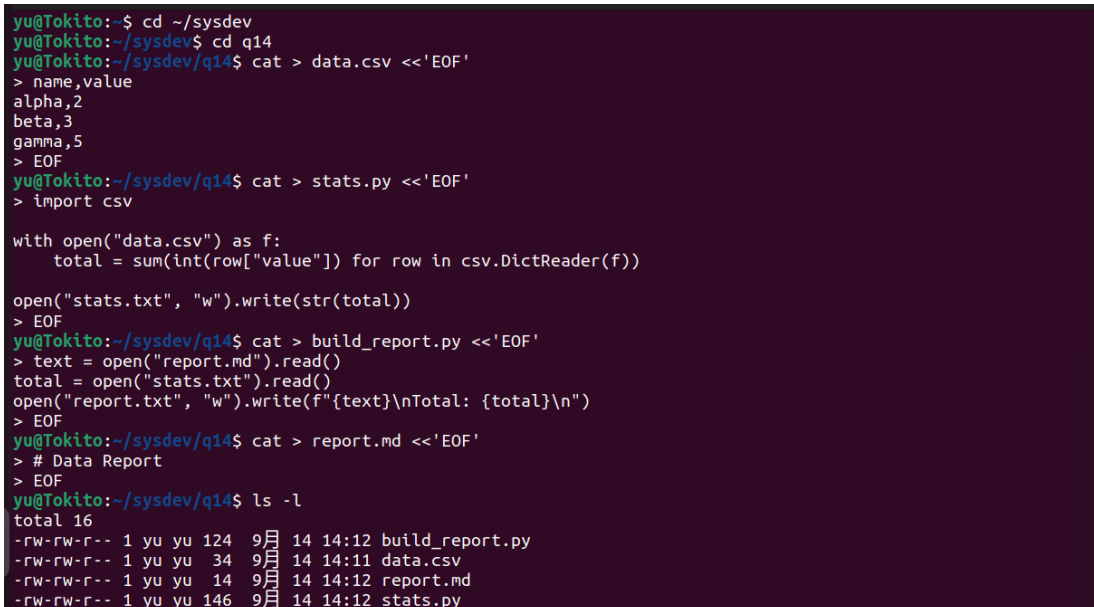
4. 在输入变化后只重建受影响的中间产物和最终产物。
5. 使用 `clean` 删除生成物但保留源文件。

3.2.2 源文件与 Makefile

按照题目给定内容创建 `data.csv`、`stats.py`、`build_report.py` 和 `report.md`。数据总和为 $2 + 3 + 5 = 10$ 。Makefile 的核心内容如下：

```
1 .PHONY: all clean
2
3 all: report.txt
4
5 stats.txt: data.csv stats.py
6     python3 stats.py
7
8 report.txt: report.md build_report.py stats.txt
9     python3 build_report.py
10
11 clean:
12     rm -f stats.txt report.txt
```

其中 `stats.txt` 同时依赖数据文件和统计脚本；`report.txt` 同时依赖 Markdown 模板、报告生成脚本和已经生成的 `stats.txt`。只要依赖文件比目标文件更新，Make 就会重新执行对应配方；否则输出 “Nothing to be done”。



```
yu@Tokito:~$ cd ~/sysdev
yu@Tokito:~/sysdev$ cd q14
yu@Tokito:~/sysdev/q14$ cat > data.csv <<'EOF'
> name,value
alpha,2
beta,3
gamma,5
> EOF
yu@Tokito:~/sysdev/q14$ cat > stats.py <<'EOF'
> import csv

with open("data.csv") as f:
    total = sum(int(row["value"]) for row in csv.DictReader(f))

open("stats.txt", "w").write(str(total))
> EOF
yu@Tokito:~/sysdev/q14$ cat > build_report.py <<'EOF'
> text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
> EOF
yu@Tokito:~/sysdev/q14$ cat > report.md <<'EOF'
> # Data Report
> EOF
yu@Tokito:~/sysdev/q14$ ls -l
total 16
-rw-rw-r-- 1 yu yu 124  9月 14 14:12 build_report.py
-rw-rw-r-- 1 yu yu  34  9月 14 14:11 data.csv
-rw-rw-r-- 1 yu yu  14  9月 14 14:12 report.md
-rw-rw-r-- 1 yu yu 146  9月 14 14:12 stats.py
```

图 6: Q14 创建给定数据、统计脚本、报告脚本与 Markdown 源文件

3.2.3 增量构建验证

首次运行 `make` 时依次执行 `python3 stats.py` 和 `python3 build_report.py`，生成 `stats.txt` 与 `report.txt`。立即再次运行 `make` 时，两个目标都早于依赖，因此没有执行任何脚本。

随后执行 `touch data.csv` 改变数据文件的修改时间，再次运行 `make` 时两个配方都重新执行。这说明 `stats.txt` 已正确声明对 `data.csv` 的依赖，而变化又会继续传递到最终报告。运行 `make clean` 后，两个生成物被删除，四个源文件仍然保留。

```
1 $ make
2 python3 stats.py
3 python3 build_report.py
4
5 $ cat stats.txt
6 10
7
8 $ cat report.txt
9 # Data Report
10 Total: 10
11
12 $ make
13 make: Nothing to be done for 'all'.
14
15 $ touch data.csv
16 $ make
17 python3 stats.py
18 python3 build_report.py
19
20 $ make clean
21 rm -f stats.txt report.txt
```

```
yu@Tokito:~/sysdev/q14$ cat > Makefile <<'EOF'
> .PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
python3 stats.py

report.txt: report.md build_report.py stats.txt
python3 build_report.py

clean:
rm -f stats.txt report.txt
> EOF
yu@Tokito:~/sysdev/q14$ cat -n Makefile
1 .PHONY: all clean
2
3 all: report.txt
4
5 stats.txt: data.csv stats.py
6 python3 stats.py
7
8 report.txt: report.md build_report.py stats.txt
9 python3 build_report.py
10
11 clean:
12 rm -f stats.txt report.txt
yu@Tokito:~/sysdev/q14$ make
python3 stats.py
python3 build_report.py
yu@Tokito:~/sysdev/q14$ cat stats.txt
10yu@Tokito:~/sysdev/q14$ cat report.txt
# Data Report

Total: 10
yu@Tokito:~/sysdev/q14$ make
make: Nothing to be done for 'all'.
yu@Tokito:~/sysdev/q14$ touch data.csv
yu@Tokito:~/sysdev/q14$ make
python3 stats.py
python3 build_report.py
yu@Tokito:~/sysdev/q14$ make clean
rm -f stats.txt report.txt
yu@Tokito:~/sysdev/q14$ ls -l
total 20
-rw-rw-r-- 1 yu yu 124  9月 14 14:12 build_report.py
-rw-rw-r-- 1 yu yu  34  9月 14 14:14 data.csv
-rw-rw-r-- 1 yu yu 193  9月 14 14:13 Makefile
-rw-rw-r-- 1 yu yu  14  9月 14 14:12 report.md
-rw-rw-r-- 1 yu yu 146  9月 14 14:12 stats.py
yu@Tokito:~/sysdev/q14$ make clean
rm -f stats.txt report.txt
yu@Tokito:~/sysdev/q14$ make
python3 stats.py
python3 build_report.py
yu@Tokito:~/sysdev/q14$ make
make: Nothing to be done for 'all'.
yu@Tokito:~/sysdev/q14$ touch data.csv
yu@Tokito:~/sysdev/q14$ make
python3 stats.py
python3 build_report.py
yu@Tokito:~/sysdev/q14$ make clean
rm -f stats.txt report.txt
```

图 7: Q14 首次构建、无改动跳过、touch 后重建与 clean 验证

3.2.4 实验结果与提交记录

清理后从零运行 `make` 可再次生成 `stats.txt` 和 `report.txt`，内容分别为 10 与包含 Total: 10 的 Markdown 报告。随后再次运行 `make` 不再执行配方。实验最终将 5 个源文件提交为 1f95941，提交信息为 feat(q14): add make build pipeline。

```
yu@Tokito:~/sysdev/q14$ git status
fatal: not a git repository (or any of the parent directories): .git
yu@Tokito:~/sysdev/q14$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:     git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:     git branch -m <name>
Initialized empty Git repository in /home/yu/sysdev/q14/.git/
yu@Tokito:~/sysdev/q14$ git add data.csv stats.py build_report.py report.md Makefile
yu@Tokito:~/sysdev/q14$ git commit -m "feat(q14): add make build pipeline"
[master (root-commit) 1f95941] feat(q14): add make build pipeline
5 files changed, 26 insertions(+)
create mode 100644 Makefile
create mode 100644 build_report.py
create mode 100644 data.csv
create mode 100644 report.md
create mode 100644 stats.py
```

图 8: Q14 初始化 Git、添加源文件并完成聚焦提交

```
yu@Tokito:~/sysdev/q14$ ls -l
total 20
-rw-rw-r-- 1 yu yu 124  9月 14 14:12 build_report.py
-rw-rw-r-- 1 yu yu  34  9月 14 14:14 data.csv
-rw-rw-r-- 1 yu yu 193  9月 14 14:13 Makefile
-rw-rw-r-- 1 yu yu  14  9月 14 14:12 report.md
-rw-rw-r-- 1 yu yu 146  9月 14 14:12 stats.py
yu@Tokito:~/sysdev/q14$ cat Makefile
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: report.md build_report.py stats.txt
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
yu@Tokito:~/sysdev/q14$ make clean
rm -f stats.txt report.txt
yu@Tokito:~/sysdev/q14$ make
python3 stats.py
python3 build_report.py
yu@Tokito:~/sysdev/q14$ ls -l stats.txt report.txt
-rw-rw-r-- 1 yu yu 25  9月 14 14:18 report.txt
-rw-rw-r-- 1 yu yu  2  9月 14 14:18 stats.txt
yu@Tokito:~/sysdev/q14$ cat stats.txt
10yu@Tokito:~/sysdev/q14$ cat report.txt
# Data Report

Total: 10
yu@Tokito:~/sysdev/q14$ make
make: Nothing to be done for 'all'.
yu@Tokito:~/sysdev/q14$ touch data.csv
yu@Tokito:~/sysdev/q14$ make
python3 stats.py
python3 build_report.py
yu@Tokito:~/sysdev/q14$ make clean
rm -f stats.txt report.txt
yu@Tokito:~/sysdev/q14$ ls -l
total 20
-rw-rw-r-- 1 yu yu 124  9月 14 14:12 build_report.py
-rw-rw-r-- 1 yu yu  34  9月 14 14:18 data.csv
-rw-rw-r-- 1 yu yu 193  9月 14 14:13 Makefile
-rw-rw-r-- 1 yu yu  14  9月 14 14:12 report.md
-rw-rw-r-- 1 yu yu 146  9月 14 14:12 stats.py
```

图 9: Q14 最终产物、增量构建与 clean 后的重建结果

3.2.5 问题分析与体会

Make 不是按照命令的“感觉”决定是否构建，而是严格按照目标文件与依赖文件的时间戳判断。依赖关系写漏时，首次构建可能仍然成功，但后续修改不会触发正确重

建，因此“第一次能跑”不能证明 Makefile 正确。这个实验也解释了为什么构建系统要显式描述依赖图：只有依赖关系完整，增量构建才既快速又可靠。

3.3 实验十五：把本地 API 数据转换为可读报告

3.3.1 实验目的

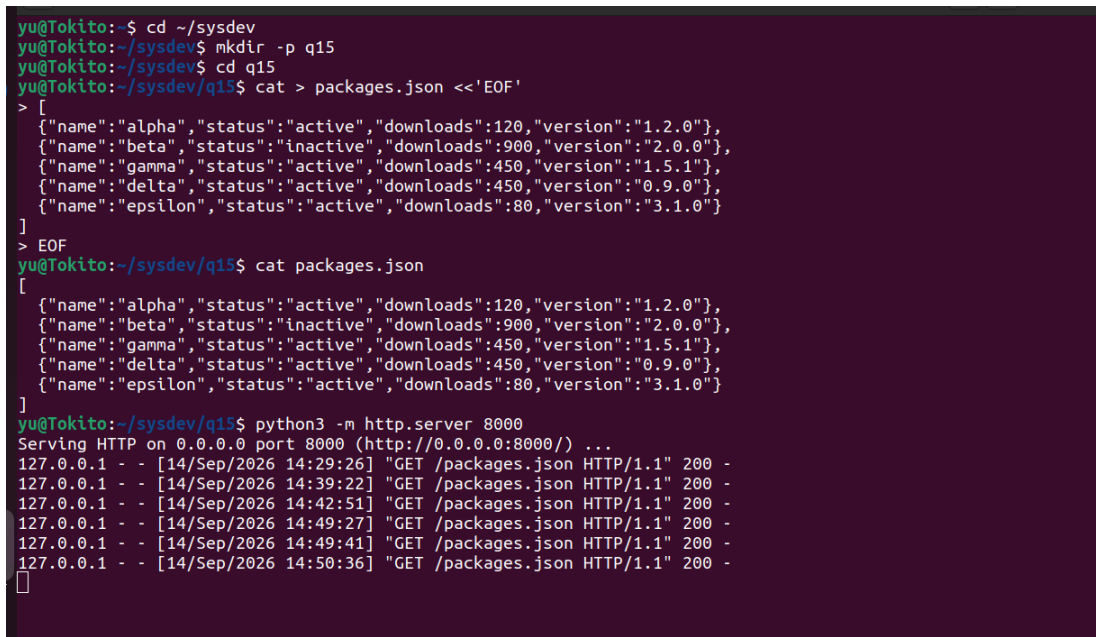
1. 使用 Python 标准库启动本地 HTTP 文件服务。
2. 使用 `curl -fsS` 按失败即退出、静默错误的方式获取 JSON。
3. 使用 `jq` 完成筛选、多条件排序与 Markdown 表格生成。
4. 将 API、命令行文本处理与报告写入组合成一个可重复脚本。

3.3.2 启动服务并获取数据

先把给定五条软件包记录保存为 `packages.json`，再在 Q15 目录执行：

```
1 python -m http.server 8000
2 curl -fsS http://127.0.0.1:8000/packages.json
```

服务终端显示多次 GET `/packages.json` HTTP/1.1 返回 200，说明 `curl` 与 `jq` 面对的是真实 HTTP 请求，而不是直接读取本地文件。本地文件方式虽然更简单，但无法验证 URL、HTTP 状态码和 `curl` 的失败传播行为。



```
yu@Tokito:~$ cd ~/sysdev
yu@Tokito:~/sysdev$ mkdir -p q15
yu@Tokito:~/sysdev$ cd q15
yu@Tokito:~/sysdev/q15$ cat > packages.json <<'EOF'
> [
  {"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
  {"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
  {"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
  {"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
  {"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
]
> EOF
yu@Tokito:~/sysdev/q15$ cat packages.json
[
  {"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
  {"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
  {"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
  {"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
  {"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
]
yu@Tokito:~/sysdev/q15$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [14/Sep/2026 14:29:26] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 14:39:22] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 14:42:51] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 14:49:27] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 14:49:41] "GET /packages.json HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 14:50:36] "GET /packages.json HTTP/1.1" 200 -
□
```

图 10: Q15 启动本地 HTTP 服务并成功返回 `packages.json`

3.3.3 筛选、排序与报告脚本

用于筛选和排序的 jq 表达式为：

```
1 .[] |  
2 select(.status == "active" and .downloads >= 100) |  
3 [.downloads, .name]
```

实际脚本把结果按 downloads 降序、name 升序排列，并转换成三列表格：

```
1 #!/usr/bin/env bash  
2 set -euo pipefail  
3  
4 url="http://127.0.0.1:8000/packages.json"  
5  
6 {  
7     echo "# Package Report"  
8     echo  
9 }  
10 curl -fsS "$url" |  
11 jq -r '  
12     ["name", "version", "downloads"],  
13     ([.[] |  
14         select(.status == "active" and .downloads >= 100)] |  
15         sort_by(-.downloads, .name) |  
16         map([.name, .version, .downloads]))[] |  
17     "| \(.name) | \(.version) | \(.downloads) |"  
18 ' > summary.md  
19  
20 echo "generated summary.md"
```

脚本使用 `set -euo pipefail`，因此 `curl` 的 HTTP 失败、`jq` 解析失败或后续命令失败都会使整个脚本返回非零状态，而不是继续写入不完整报告。

```
yu@Tokito:~$ cd ~/sysdev/q15
yu@Tokito:~/sysdev/q15$ curl -fsS http://127.0.0.1:8000/packages.json
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
yu@Tokito:~/sysdev/q15$ jq --version
jq-1.6
yu@Tokito:~/sysdev/q15$ curl -fsS http://127.0.0.1:8000/packages.json \
| jq '[][ | select(.status == "active" and .downloads >= 100)] | sort_by([-.downloads, .name])'
[
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  }
]
yu@Tokito:~/sysdev/q15$ cat > api_report.sh <<'EOF'
> #!/usr/bin/env bash

set -euo pipefail

url="http://127.0.0.1:8000/packages.json"

curl -fsS "$url" \
| jq -r '
[
  "# Package Report",
  "",
  "| name | version | downloads |",
  "|---|---:|---:|",
  (
    | map(select(.status == "active" and .downloads >= 100))
    | sort_by([-.downloads, .name])
    | .[]
    | "| \(.name) | \(.version) | \(.downloads) |"
  )
]
| .[]
' > summary.md

echo "generated summary.md"
> EOF
yu@Tokito:~/sysdev/q15$ chmod +x api_report.sh
```

图 11: Q15 使用 curl 获取 JSON 并用 jq 验证筛选和排序表达式

```
yu@Tokito:~/sysdev/q15$ chmod +x api_report.sh
yu@Tokito:~/sysdev/q15$ cat -n api_report.sh
 1  #!/usr/bin/env bash
 2
 3  set -euo pipefail
 4
 5  url="http://127.0.0.1:8000/packages.json"
 6
 7  curl -fsS "$url" \
 8  | jq -r '
 9  [
10      "# Package Report",
11      "",
12      "| name | version | downloads |",
13      "|---|---:|---:|",
14      (.
15        | map(select(.status == "active" and .downloads >= 100))
16        | sort_by([-.downloads, .name])
17        | .[]
18        | "| \(.name) | \(.version) | \(.downloads) |"
19      )
20    ]
21    | .[]
22    ' > summary.md
23
24  echo "generated summary.md"
yu@Tokito:~/sysdev/q15$ ./api_report.sh
generated summary.md
yu@Tokito:~/sysdev/q15$ cat summary.md
# Package Report

| name | version | downloads |
|---|---:|---:|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 12: Q15 api_report.sh 内容与 summary.md 生成结果

3.3.4 实验结果

满足条件的数据共有三条，排序结果为：

表 2: Q15 筛选与排序结果

name	version	downloads
delta	0.9.0	450
gamma	1.5.1	450
alpha	1.2.0	120

beta 因状态为 inactive 被过滤，epsilon 因下载量为 80 被过滤。delta 与 gamma 下载量相同，最终按照 name 升序，因此 delta 位于 gamma 之前。执行脚本后成功生成 summary.md。

```

yu@Tokito:~/sysdev/q15$ curl -fsS http://127.0.0.1:8000/packages.json >/dev/null
yu@Tokito:~/sysdev/q15$ ./api_report.sh
generated summary.md
yu@Tokito:~/sysdev/q15$ cat summary.md
# Package Report

| name | version | downloads |
|---|---|---|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
yu@Tokito:~/sysdev/q15$ ls -l packages.json api_report.sh summary.md
-rwxrwxr-x 1 yu yu 502 9月 14 14:50 api_report.sh
-rw-rw-r-- 1 yu yu 365 9月 14 14:26 packages.json
-rw-rw-r-- 1 yu yu 137 9月 14 14:54 summary.md

```

图 13: Q15 最终执行 api_report.sh 并生成排序后的 Markdown 报告

3.3.5 问题分析与体会

该实验的关键不只是“写出一个能运行的 jq 表达式”，而是同时保证 HTTP 获取可靠、排序规则明确、输出格式稳定。把 curl 的输出直接传给 jq，再由脚本重定向到 Markdown 文件，形成了清晰的数据流水线。-fsS 让 HTTP 错误不会被误当成正常 JSON，pipefail 则避免前半段失败却被后续命令掩盖。

3.4 实验十六：修复并交付一个陌生的小型工具仓库

3.4.1 实验目的

1. 在真实仓库中定位测试、源码和检查脚本之间的关系。
2. 主动制造质量问题并确认测试能够捕获错误。
3. 修复实现后重新运行本地质量门禁。
4. 使用 Makefile 统一执行检查、构建和清理。
5. 计算 Wheel 的 SHA-256，并以聚焦提交交付改动。

3.4.2 检查源码和测试

Q16 复制 Q13 后先重新初始化 Git，并提交初始版本 d76a88b。随后查看测试与源码，确认两个测试分别验证空白姓名退出码和正常姓名输出。

```

1 import sys
2 import pytest
3 from greetlab.cli import main
4
5
6 def test_blank_name_exits_with_code_2(monkeypatch):
7     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
8     with pytest.raises(SystemExit) as exc_info:

```

```
9     main()
10     assert exc_info.value.code == 2
11
12
13 def test_normal_name_prints_greeting(monkeypatch, capsys):
14     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Alice"])
15     main()
16     captured = capsys.readouterr()
17     assert captured.out == "Hello, Alice!\n"
```

对应实现会在打印问候语前检查姓名是否为空白：

```
1 import argparse
2
3
4 def main():
5     parser = argparse.ArgumentParser()
6     parser.add_argument("--name", required=True)
7     args = parser.parse_args()
8
9     if not args.name.strip():
10         parser.error("--name cannot be empty or whitespace")
11
12     print(f"Hello, {args.name}!")
```

```

yu@Tokito:~$ cd ~/sysdev
yu@Tokito:~/sysdev$ cd q16
yu@Tokito:~/sysdev/q16$ rm -rf .git
yu@Tokito:~/sysdev/q16$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/yu/sysdev/q16/.git/
yu@Tokito:~/sysdev/q16$ git add .
yu@Tokito:~/sysdev/q16$ git commit -m "初始化 q16 项目"
[master (root-commit) d76a88b] 初始化 q16 项目
13 files changed, 144 insertions(+)
create mode 100644 .gitignore
create mode 100644 ai_log.md
create mode 100755 check.sh
create mode 100644 pyproject.toml
create mode 100644 q10/.gitignore
create mode 100644 q10/ai_log.md
create mode 100644 q10/pyproject.toml
create mode 100644 q10/src/greetlab/__init__.py
create mode 100644 q10/src/greetlab/cli.py
create mode 100644 q10/tests/test_cli.py
create mode 100644 src/greetlab/__init__.py
create mode 100644 src/greetlab/cli.py
create mode 100644 tests/test_cli.py
yu@Tokito:~/sysdev/q16$ git status
On branch master
nothing to commit, working tree clean
yu@Tokito:~/sysdev/q16$ source .venv/bin/activate
(.venv) yu@Tokito:~/sysdev/q16$ ./check.sh
8 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.13s =====
(.venv) yu@Tokito:~/sysdev/q16$ cat check.sh
#!/usr/bin/env bash
set -euo pipefail

python -m ruff format --check .
python -m ruff check .
python -m pytest
(.venv) yu@Tokito:~/sysdev/q16$ git add check.sh
(.venv) yu@Tokito:~/sysdev/q16$ git commit -m "修正检查脚本"
On branch master
nothing to commit, working tree clean
(.venv) yu@Tokito:~/sysdev/q16$ grep -rniE 'hello|greet|问候' --exclude-dir=.git --exclude-dir=.venv .
./pyproject.toml:6:name = "greetlab"
./pyproject.toml:8:description = "A small command-line greeting tool"
./pyproject.toml:13:sd-greet = "greetlab.cli:main"
./ai_log.md:1:目标：空白 name 必须以 SystemExit(2) 结束，并保留正常问候行为。
grep: ./tests/__pycache__/test_cli.cpython-310-pytest-9.1.1.pyc: binary file matches
./tests/test_cli.py:5:from greetlab.cli import main

```

图 14: Q16 初始化仓库、执行 check.sh 并检查仓库状态

```
grep: ./ruff_cache/0.16.7/12110291425602827135: binary file matches
(.venv) yu@Tokito:~/sysdev/q16$ ./check.sh
8 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.13s =====
(.venv) yu@Tokito:~/sysdev/q16$ sed -n '1,240p' tests/test_cli.py
import sys

import pytest

from greetlab.cli import main

def test_blank_name_exits_with_code_2(monkeypatch):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])

    with pytest.raises(SystemExit) as exc_info:
        main()

    assert exc_info.value.code == 2

def test_normal_name_prints_greeting(monkeypatch, capsys):
    monkeypatch.setattr(sys, "argv", ["sd-greet", "--name", "Alice"])

    main()

    captured = capsys.readouterr()
    assert captured.out == "Hello, Alice!\n"
(.venv) yu@Tokito:~/sysdev/q16$ nl -ba src/greetlab/cli.py
1 import argparse
2
3
4 def main():
5     p = argparse.ArgumentParser()
6     p.add_argument("--name", required=True)
7     a = p.parse_args()
8     if not a.name.strip():
9         p.error("--name cannot be empty or whitespace")
10    print(f"Hello, {a.name}!")
```

图 15: Q16 查看测试文件和源码实现

3.4.3 故障复现与修复

为了验证测试不是“只会通过”，把问候语实现临时改成始终返回字面量 `Hello, name!`。再次执行 `check.sh` 后，正常姓名测试失败，证明测试确实检查了实际姓名插值，而不是只检查退出状态。

恢复正确实现后，Ruff 格式检查与静态检查重新通过，pytest 显示 2 个测试全部通过。这里没有修改测试去迎合错误实现，而是修复源码并保留原有断言。

```

10 print(f'Hello, {a.name}!')
(.venv) yu@Tokito:~/sysdev/q16$ nl -ba tests/test_cli.py
1 import sys
2
3 import pytest
4
5 from greetlab.cli import main
6
7
8 def test_blank_name_exits_with_code_2(monkeypatch):
9     monkeypatch.setattr(sys, "argv", ["sd-greet", "--name", ""])
10
11     with pytest.raises(SystemExit) as exc_info:
12         main()
13
14     assert exc_info.value.code == 2
15
16
17 def test_normal_name_prints_greeting(monkeypatch, capsys):
18     monkeypatch.setattr(sys, "argv", ["sd-greet", "--name", "Alice"])
19
20     main()
21
22     captured = capsys.readouterr()
23     assert captured.out == "Hello, Alice!\n"
(.venv) yu@Tokito:~/sysdev/q16$ ./check.sh
8 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.22s =====
(.venv) yu@Tokito:~/sysdev/q16$ nano src/greetlab/cli.py
(.venv) yu@Tokito:~/sysdev/q16$ git diff -- src/greetlab/cli.py
diff --git a/src/greetlab/cli.py b/src/greetlab/cli.py
index 53750bc..84f5a6e 100644
--- a/src/greetlab/cli.py
+++ b/src/greetlab/cli.py
@@ -7,4 +7,4 @@ def main():
     a = p.parse_args()
     if not a.name.strip():
         p.error("--name cannot be empty or whitespace")
-    print(f'Hello, {a.name}!')
+    print(f'Hello, name!')
(.venv) yu@Tokito:~/sysdev/q16$ ./check.sh
8 files already formatted
F541 [*] f-string without any placeholders
--> src/greetlab/cli.py:10:11
8 |     if not a.name.strip():
9 |         p.error("--name cannot be empty or whitespace")
10 |     print(f'Hello, name!')
    ~~~~~
help: Remove extraneous `f` prefix
9 |         p.error("--name cannot be empty or whitespace")
-    print(f'Hello, name!')
10 +    print("Hello, name!")

Found 1 error.
[*] 1 fixable with the `--fix` option.

```

图 16: Q16 故意修改问候语后 check.sh 捕获测试失败

```
[*] 1 fixable with the '--fix' option.
(.venv) yu@Tokito:~/sysdev/q16$ nano src/greetlab/cli.py
(.venv) yu@Tokito:~/sysdev/q16$ grep -n "print" src/greetlab/cli.py
10:     print(f"Hello, {a.name}!")
(.venv) yu@Tokito:~/sysdev/q16$ ./check.sh
8 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py ..                                     [100%]

===== 2 passed in 0.12s =====
(.venv) yu@Tokito:~/sysdev/q16$ git status --short
(.venv) yu@Tokito:~/sysdev/q16$ git diff -- src/greetlab/cli.py
(.venv) yu@Tokito:~/sysdev/q16$ git add src/greetlab/cli.py
(.venv) yu@Tokito:~/sysdev/q16$ git commit -m "修复问候语输出"
On branch master
nothing to commit, working tree clean
(.venv) yu@Tokito:~/sysdev/q16$ git log --oneline --decorate -5
d76a88b (HEAD -> master) 初始化 q16 项目
(.venv) yu@Tokito:~/sysdev/q16$ nano Makefile
(.venv) yu@Tokito:~/sysdev/q16$ cat -A Makefile
.PHONY: check build clean$
$
check:$
^I./check.sh$
$
build:$
^Ipthon -m build$
$
clean:$
^Irm -rf build dist *.egg-info$
(.venv) yu@Tokito:~/sysdev/q16$ python -m pip install build
Requirement already satisfied: build in ./venv/lib/python3.10/site-packages (1.6.1)
Requirement already satisfied: packaging>=24.0 in ./venv/lib/python3.10/site-packages (from build) (26.3)
Requirement already satisfied: pyproject_hooks in ./venv/lib/python3.10/site-packages (from build) (1.2.0)
Requirement already satisfied: tomli>=1.1.0 in ./venv/lib/python3.10/site-packages (from build) (2.4.1)
(.venv) yu@Tokito:~/sysdev/q16$ make check
./check.sh
8 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py ..                                     [100%]

===== 2 passed in 0.08s =====
```

图 17: Q16 修复问候语并重新运行检查与测试

3.4.4 Makefile、构建与哈希

补充的 Makefile 包含 check、build 和 clean:

```
1 .PHONY: check build clean
2
3 check:
4     ./check.sh
5
6 build:
7     python -m build
8
9 clean:
10    rm -rf build dist *.egg-info
```

make check 最终显示 8 个文件已经格式化、Ruff 全部通过、2 个测试通过，并且 shell 退出码为 0。随后执行 make build，成功生成源码包和 greetlab-0.1.0-py3-none-any.whl。

```

1 $ make check
2 8 files already formatted
3 All checks passed!
4 2 passed in 0.16s
5
6 $ echo $?
7 0
8
9 $ make build
10 Successfully built greetlab-0.1.0.tar.gz
11 Successfully built greetlab-0.1.0-py3-none-any.whl

```

```

===== 2 passed in 0.08s =====
(.venv) yu@Tokito:~/sysdev/q16$ ./check.sh
8 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py ..                                     [100%]

===== 2 passed in 0.16s =====
(.venv) yu@Tokito:~/sysdev/q16$ echo $?
0
(.venv) yu@Tokito:~/sysdev/q16$ make build
python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=61
* Getting build dependencies for sdist...
running egg_info
writing src/greetlab.egg-info/PKG-INFO
writing dependency links to src/greetlab.egg-info/dependency_links.txt
writing entry points to src/greetlab.egg-info/entry_points.txt
writing requirements to src/greetlab.egg-info/requirements.txt
writing top-level names to src/greetlab.egg-info/top_level.txt
reading manifest file 'src/greetlab.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab.egg-info/SOURCES.txt'
* Installed build dependency versions:

```

图 18: Q16 make check 返回 0 并由 make build 生成 Wheel

Wheel 生成后使用 SHA-256 记录构建产物。一次构建得到：

```

1 $ sha256sum dist/*.whl
2 d0afd1fee7d1db3d0a353845a03ed8b3a309be7ce6d63ba2e2490bf325928c70 dist/greetlab
  -0.1.0-py3-none-any.whl

```

清理后重新构建时哈希变为：

```

1 $ sha256sum dist/*.whl
2 33af7036156979f83bbaa36a6b9a23ad8a8c2c8fa929fda5d83dba72c126dd21 dist/greetlab
  -0.1.0-py3-none-any.whl

```

原因是普通 Python Wheel 内部可能记录构建时间，因而即使是同一份源代码也未必得到逐字节相同的归档。SHA-256 在这里用于固定并核对交付文件，而不是证明构建具有完全的二进制可复现性。

```
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab-0.1.0.tar.gz and greetlab-0.1.0-py3-none-any.whl
(.venv) yu@Tokito:~/sysdev/q16$ find dist -maxdepth 1 -type f -name '*.whl' -print
dist/greetlab-0.1.0-py3-none-any.whl
(.venv) yu@Tokito:~/sysdev/q16$ sha256sum dist/*.whl
d0afd1fee7d1db3d0a353845a03ed8b3a309be7ce6d63ba2e2490bf325928c70 dist/greetlab-0.1.0-py3-none-any.whl
(.venv) yu@Tokito:~/sysdev/q16$ sha256sum dist/*.whl | tee wheel.sha256
d0afd1fee7d1db3d0a353845a03ed8b3a309be7ce6d63ba2e2490bf325928c70 dist/greetlab-0.1.0-py3-none-any.whl
(.venv) yu@Tokito:~/sysdev/q16$ git add Makefile wheel.sha256
(.venv) yu@Tokito:~/sysdev/q16$ git commit -m "添加检查与构建流程"
[master 9da5a0c] 添加检查与构建流程
2 files changed, 11 insertions(+)
create mode 100644 Makefile
create mode 100644 wheel.sha256
(.venv) yu@Tokito:~/sysdev/q16$ make clean
rm -rf build dist *.egg-info
(.venv) yu@Tokito:~/sysdev/q16$ make check
./check.sh
8 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/sysdev/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.14s =====
(.venv) yu@Tokito:~/sysdev/q16$ make build
python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
- setuptools>=61
* Getting build dependencies for sdist...
running egg_info
writing src/greetlab.egg-info/PKG-INFO
writing dependency_links to src/greetlab.egg-info/dependency_links.txt
writing entry_points to src/greetlab.egg-info/entry_points.txt
writing requirements to src/greetlab.egg-info/requirements.txt
writing top-level names to src/greetlab.egg-info/top_level.txt
reading manifest file 'src/greetlab.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab.egg-info/SOURCES.txt'
* Installed build dependency versions:
```

图 19: Q16 Wheel 哈希记录、Makefile 提交、clean 与重新检查

```
adding 'greetlab-0.1.0.dist-info/entry_points.txt'
adding 'greetlab-0.1.0.dist-info/top_level.txt'
adding 'greetlab-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab-0.1.0.tar.gz and greetlab-0.1.0-py3-none-any.whl
(.venv) yu@Tokito:~/sysdev/q16$ sha256sum dist/*.whl
33af7036156979f83bbaa36a6b9a23ad8a8c2c8fa929fda5d83dba72c126dd21 dist/greetlab-0.1.0-py3-none-any.whl
9da5a0c (HEAD -> master) 添加检查与构建流程
d76a88b 初始化 q16 项目
(.venv) yu@Tokito:~/sysdev/q16$ git status
On branch master
nothing to commit, working tree clean
(.venv) yu@Tokito:~/sysdev/q16$
```

图 20: Q16 最终重建、Wheel 哈希和工作区 clean 状态

3.4.5 实验结果与体会

最终提交包含检查和构建流程，提交编号包括初始提交 d76a88b 与后续构建流程提交 9da5a0c。make clean 后重新执行 make check 仍能通过，说明清理生成物不会破

坏源码与测试。综合测试的核心不是“尽快修改一行代码”，而是完整经历故障复现、修复、验证、构建、哈希记录和提交这一条闭环。

4 课后练习

5 课后练习一：配置格式化器、linter 与 pre-commit 钩子

练习创建了 `demo.py`、`.pre-commit-config.yaml` 和 Git 仓库。首先安装 pre-commit 钩子：

```
1 pre-commit install
2 pre-commit run --all-files
```

配置文件的最小形式为：

```
1 repos:
2   - repo: https://github.com/astral-sh/ruff-pre-commit
3     rev: v0.9.10
4     hooks:
5       - id: ruff-format
6       - id: ruff
```

初始示例中保留了一个未使用变量：

```
1 def calc(a, b):
2     x = a + b
3     return a + b
4
5
6 def hello():
7     print("hello world")
```

```
(.venv) yu@Tokito:~/lint-demo$ cat > demo.py <<'EOF'
> def calc(a,b):
>     x= a + b
>     unused_var = 123
>     return x
>
> def hello():
>     print( "hello world" )
> EOF
(.venv) yu@Tokito:~/lint-demo$ cat > .pre-commit-config.yaml <<'EOF'
> repos:
> - repo: https://github.com/astral-sh/ruff-pre-commit
>   rev: v0.9.10
>   hooks:
>     - id: ruff-format
>     - id: ruff
> EOF
(.venv) yu@Tokito:~/lint-demo$ pre-commit install
pre-commit installed at .git/hooks/pre-commit
(.venv) yu@Tokito:~/lint-demo$ ruff format demo.py
1 file reformatted
(.venv) yu@Tokito:~/lint-demo$ ruff check demo.py
F841 Local variable `unused_var` is assigned to but never used
--> demo.py:3:5
1 | def calc(a, b):
2 |     x = a + b
3 |     unused_var = 123
  |     ^^^^^^^^^^^
4 |     return x
help: Remove assignment to unused variable `unused_var`

Found 1 error.
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option).
(.venv) yu@Tokito:~/lint-demo$ git add demo.py
(.venv) yu@Tokito:~/lint-demo$ git commit -m "test precommit"
[INFO] Initializing environment for https://github.com/astral-sh/ruff-pre-commit.
```

图 21: 课后练习一：创建 pre-commit 配置并检查格式与未使用变量

5.1 AI Agent 运行成果（无截图，正文留档）

给 AI agent 的约束如下：先运行 Ruff 并读取完整输出；只修复真实质量问题，不通过全局 ignore 掩盖错误；每完成一轮修改都必须再次运行检查；最后报告修改文件和验证命令。

AI agent 的文本运行记录为：

```
1 [AI agent] 读取到的关键错误:
2 demo.py:3:5: F841 Local variable `x` is assigned to but never used
3
4 [AI agent] 修复策略: 手动删除没有参与返回值计算的 `x = a + b`, 保留 `return a + b`。
5
6 $ ruff check demo.py
7 All checks passed!
8
9 $ ruff format demo.py
10 1 file reformatted
11
12 $ pre-commit run --all-files
13 ruff-format.....Passed
```

```

14 ruff.....Passed
15
16 $ git commit -m "test pre-commit"
17 [INFO] Initializing environment for astral-sh/ruff-pre-commit.
18 ruff-format.....Passed
19 ruff.....Passed
20 [master] test pre-commit
21 1 file changed, 1 insertion(+), 1 deletion(-)

```

这里的 AI 运行成果不是一段无法核验的自然语言说明，而是“错误定位、修改动作、重新运行、钩子放行”的闭环。人工检查重点是确认删除的变量确实是死代码，而不是让 AI 通过修改测试或关闭 Ruff 规则来制造假通过。

6 课后练习二：单元测试、覆盖率报告与 AI 辅助提升

练习为 calc 和 hello 编写测试，并运行：

```
1 pytest test_demo.py --cov=demo --cov-report=html
```

```

yu@Tokito:~$ cd ~/lint-demo
yu@Tokito:~/lint-demo$ cat > test_demo.py <<'EOF'
> from demo import calc,hello

def test_calc():
    assert calc(1,2) == 3

def test_calc_neg():
    assert calc(-1, -5) == -6
> EOF
yu@Tokito:~/lint-demo$ pytest test_demo.py --cov=demo --cov-report=html
Command 'pytest' not found, did you mean:
  command 'ptest' from deb pbuilder-scripts (22)
  command 'ptest' from deb parmetis-test (4.0.3-5build1)
  command 'yptest' from deb yp-tools (4.2.3-3)
Try: sudo apt install <deb name>
yu@Tokito:~/lint-demo$ source .venv/bin/activate
(.venv) yu@Tokito:~/lint-demo$ pytest test_demo.py --cov=demo --cov-report=html
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yu/lint-demo
plugins: anyio-4.15.1, cov-7.1.0
collected 2 items

test_demo.py .. [100%]

===== tests coverage =====
coverage: platform linux, python 3.10.12-final-0

Coverage HTML written to dir htmlcov
===== 2 passed in 0.15s =====
(.venv) yu@Tokito:~/lint-demo$

```

图 22: 课后练习二：编写单元测试并生成 HTML 覆盖率报告

6.1 AI Agent 运行成果（无截图，正文留档）

第一次覆盖率运行得到 83%，详细报告中 demo.py 共 6 条语句，其中 1 条未覆盖。给 AI agent 的要求是：不能只看总百分比，必须先运行带逐行缺口的覆盖率命令，再根据 missing 行补充测试，最后重新运行 pytest 和覆盖率工具。

AI agent 的关键运行结果如下：

```
1 $ python -m pytest --cov=demo --cov-report=term-missing
2 ----- coverage: platform linux, python 3.10.12 -----
3 Name Stmts Miss Cover Missing
4 -----
5 demo.py 6 1 83% 8
6 -----
7 TOTAL 6 1 83%
8 2 passed in 0.15s
9
10 [AI agent] 根据 Missing=8 定位到 `return a + b` 分支，
11 新增正向数值测试，不修改生产代码。
12
13 $ python -m pytest --cov=demo --cov-report=term-missing
14 Name Stmts Miss Cover Missing
15 -----
16 demo.py 6 0 100%
17 -----
18 TOTAL 6 0 100%
19 4 passed in 0.11s
20 Coverage HTML written to dir htmlcov
```

AI 生成的测试是否有效，关键不在于它新增了多少行，而在于新增断言是否覆盖真实分支、是否符合输入边界，以及是否在没有修改业务代码的前提下提高覆盖率。该实验中，人工复核确认新增测试只是补充算术结果断言，没有通过 mock 或跳过逻辑虚增覆盖。

7 课后练习三：配置持续集成并在每次 push 时运行

练习脚本 ci.sh 按顺序执行格式检查、lint 和带覆盖率的测试：

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 echo "ruff format check"
```

```
5 .venv/bin/ruff format --check
6
7 echo "ruff lint"
8 .venv/bin/ruff check
9
10 echo "run pytest with coverage"
11 .venv/bin/python -m pytest --cov=demo
```

持续集成工作流的核心触发条件是每次 push:

```
1 name: CI
2 on: [push, pull_request]
3 jobs:
4   quality:
5     runs-on: ubuntu-latest
6     steps:
7       - uses: actions/checkout@v4
8       - uses: actions/setup-python@v5
9         with:
10           python-version: "3.10"
11       - run: pip install -e ".[dev]"
12       - run: bash ci.sh
```



```
(.venv) yu@Tokito:~/lint-demo$ cat > ci.sh <<'EOF'
> #!/bin/bash
echo "==== 运行 ruff format 检查 ====="
.venv/bin/ruff format --check .
echo "==== 运行 ruff linter ====="
.venv/bin/ruff check .
echo "==== 运行 pytest + 覆盖率 ====="
.venv/bin/python -m pytest --cov=demo
> EOF
(.venv) yu@Tokito:~/lint-demo$ chmod +x ci.sh
(.venv) yu@Tokito:~/lint-demo$ ./ci.sh
==== 运行 ruff format 检查 =====
unformatted: File would be reformatted
--> test_demo.py:1:23
|
| - from demo import calc,hello
1 + from demo import calc, hello
2 +
3 |
4 | def test_calc():
|   -     assert calc(1,2) == 3
5 +     assert calc(1, 2) == 3
6 +
7 |
|
```

图 23: 课后练习三: 创建 CI 脚本并首次发现格式和 lint 问题

随后故意在代码中加入不良格式、lint 违规和会破坏导入的错误, 重新运行脚本。CI 首先在格式检查阶段返回非零状态, 阻止不合格代码继续进入后续步骤; 修正前一阶段后, 后续测试阶段又继续暴露 import 错误。整个流程能够准确拦截人为破坏。

```
Found 3 errors.
[*] 2 fixable with the '--fix' option (1 hidden fix can be enabled with the '--unsafe-fixes' option).
==== 运行 pytest + 覆盖率 ====
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/you/lint-demo
plugins: anyio-4.15.1, cov-7.1.0
collected 2 items

test_demo.py .. [100%]

===== tests coverage =====
coverage: platform linux, python 3.10.12-final-0
Name      Stmts  Miss  Cover
-----
demo.py      6      1    83%
TOTAL        6      1    83%
===== 2 passed in 0.13s =====
(.venv) yu@Tokito:~/lint-demo$ cat > demo.py <<'EOF'
> def calc(a,b):
    bad_var=999
    return a+b
> EOF
(.venv) yu@Tokito:~/lint-demo$ ./ci.sh
==== 运行 ruff format 检查 ====
unformatted: File would be reformatted
--> demo.py:1:12
|
| - def calc(a,b):
| -     bad_var=999
| -     return a+b
1 + def calc(a, b):
2 +     bad_var = 999
3 +     return a + b
|
|
unformatted: File would be reformatted
--> test_demo.py:1:23
|
```

图 24: 课后练习三: 覆盖率为 83% 并制造代码质量问题

```
help: Remove unused import: demo.hello
|
| - from demo import calc,hello
1 + from demo import calc
2 |
|

Found 3 errors.
[*] 2 fixable with the '--fix' option (1 hidden fix can be enabled with the '--unsafe-fixes' option).
==== 运行 pytest + 覆盖率 ====
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/you/lint-demo
plugins: anyio-4.15.1, cov-7.1.0
collected 0 items / 1 error

===== ERRORS =====
ERROR collecting test_demo.py
ImportError while importing test module '/home/you/lint-demo/test_demo.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/usr/lib/python3.10/importlib/_init_.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
test_demo.py:1: in <module>
    from demo import calc,hello
E   ImportError: cannot import name 'hello' from 'demo' (/home/you/lint-demo/demo.py)
===== short test summary info =====
ERROR test_demo.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.32s =====
(.venv) yu@Tokito:~/lint-demo$
```

图 25: 课后练习三: CI 脚本在测试收集阶段捕获导入错误

该实验说明 CI 的关键不是“额外运行一次命令”，而是把同一个可重复的质量脚本挂接到版本控制事件上。只要脚本使用非零退出码表达失败，CI 就能在合并前提供统一反馈。

8 课后练习四：使用 grep 与 semgrep 查找危险调用

先创建包含安全调用和危险调用的 Python 文件，再使用扩展正则查找：

```
1 grep -RniE 'subprocess\.Popen\(.*shell=True' .
```

初始代码中的直接调用能够被匹配：

```
1 import subprocess
2
3 bad1 = subprocess.Popen("rm -rf /tmp/abc", shell=True)
4 bad2 = subprocess.Popen(cmd, shell=True)
```

但当危险调用被拆行或通过变量拼接后，单行 grep 会失效。为了验证语义匹配能力，准备了以下变体：

```
1 import subprocess
2
3 subprocess.Popen(
4     "echo hello",
5     shell=True,
6 )
```

对应尝试执行的 semgrep 规则与命令为：

```
1 rules:
2   - id: detect-popen-shell-true
3     pattern: subprocess.Popen($CMD, shell=True)
4     message: dangerous shell=True
5     languages: [python]
6     severity: ERROR
```

```
1 .venv/bin/semgrep scan --config=shell-true.yml risk2.py
```

```
(.venv) yu@Tokito:~/lint-demo$ cat > risk.py <<'EOF'
> import subprocess

ok = subprocess.Popen(["ls","-l"])
bad1 = subprocess.Popen("rm -rf /tmp/abc", shell=True)
bad2=subprocess.Popen( cmd, shell=True )
> EOF
(.venv) yu@Tokito:~/lint-demo$ grep -nE 'subprocess\.Popen\(.*shell=True' risk.py
4:bad1 = subprocess.Popen("rm -rf /tmp/abc", shell=True)
5:bad2=subprocess.Popen( cmd, shell=True )
(.venv) yu@Tokito:~/lint-demo$ cat > risk2.py <<'EOF'
> import subprocess
res = subprocess.Popen(
    "echo hello",
    shell=True
)
> EOF
(.venv) yu@Tokito:~/lint-demo$ grep -nE 'subprocess\.Popen\(.*shell=True' risk2.py
(.venv) yu@Tokito:~/lint-demo$ cat > shell-true.yml <<'EOF'
> rules:
- id: detect-popen-shell-true
  pattern: subprocess.Popen($CMD, shell=True)
  message: dangerous shell=True
  languages: [python]
  severity: ERROR
> EOF
(.venv) yu@Tokito:~/lint-demo$ .venv/bin/semgrep scan --config=shell-true.yml risk2.py
bash: .venv/bin/semgrep: No such file or directory
(.venv) yu@Tokito:~/lint-demo$
```

图 26: 课后练习四: grep 匹配简单调用并准备 semgrep 规则

当前环境未安装 semgrep，终端返回 No such file or directory，因此本机没有完成语义扫描。理论上，semgrep 基于 AST 进行匹配，不受换行、空格和部分变量拼接形式影响，比纯文本正则更不容易被简单改写绕过；但该结论需要用成功运行的结果进一步验证。实践结论是不能直接把“grep 没匹配到”理解为“代码安全”。

9 课后练习五：使用正则查找替换 Markdown 项目符号

练习文件同时包含真正的 Markdown 项目符号、普通连字符和嵌套列表。使用如下正则只匹配“行首可含空白，随后是连字符和至少一个空白”的情况：

```
1 sed -E 's/^( [[[:space:]]* )-[[:space:]]+/\1* /' test.md > out.md
```

替换前输入为：

```
1 apple
2 - banana
3 this is dash not list
4 - nested item
```

替换后结果为：

```
1 apple
2 * banana
3 this is dash not list
4 * nested item
```

```

.yenv) yu@Tokito:~/lint-demo$ cat > shell-true.yml <<'EOF'
rules:
- id: detect-popen-shell-true
  pattern: subprocess.Popen($CMD, shell=True)
  message: dangerous shell=True
  languages: [python]
  severity: ERROR
EOF
.yenv) yu@Tokito:~/lint-demo$ .venv/bin/semgrep scan --config=shell-true.yml risk2.py
.yenv) yu@Tokito:~/lint-demo$ cat > test.md <<'EOF'
- apple
  banana
text - this is dash not list
  - nested item
EOF
.yenv) yu@Tokito:~/lint-demo$ sed -E 's/^[[:space:]]*- /\1* /g' test.md
apple
banana
text - this is dash not list
* nested item
.yenv) yu@Tokito:~/lint-demo$ sed -E 's/^[[:space:]]*- /\1* /g' test.md > out.md
.yenv) yu@Tokito:~/lint-demo$ cat out.md
apple
banana
text - this is dash not list
* nested item

```

图 27: 课后练习五：用行首锚点和空白约束替换 Markdown 项目符号

如果直接全局替换所有 - 字符，会同时破坏普通文本中的连字符、负数和水平分隔线。正则中的 ^、字符类 [[:space:]] 和 + 分别限制了位置、缩进和分隔空白，因此只修改项目符号语法。执行替换前应先用标准输出预览，再重定向到新文件，避免覆盖原文件。

10 课后练习六：从 JSON 中捕获 name 并改用 JSON 解析器

基础样例为：

```
1 {"name": "Alyssa P. Hacker", "college": "MIT"}
```

贪婪正则 "name"\s*:\s*"(.*)" 会一直匹配到末尾，并把 "college": "MIT" 一起吞入分组。将量词改为非贪婪形式后可以止于 name 字段末尾：

```

1 import re
2
3 s = '{"name": "Alyssa P. Hacker", "college": "MIT"}'
4 pattern = r'"name"\s*:\s*"(.*)"'
5 print(re.search(pattern, s).group(1))

```

输出为：

```
1 Alyssa P. Hacker
```

当姓名包含 JSON 转义双引号时，正则会失效或捕获错误内容。允许转义字符的表达式可写成：

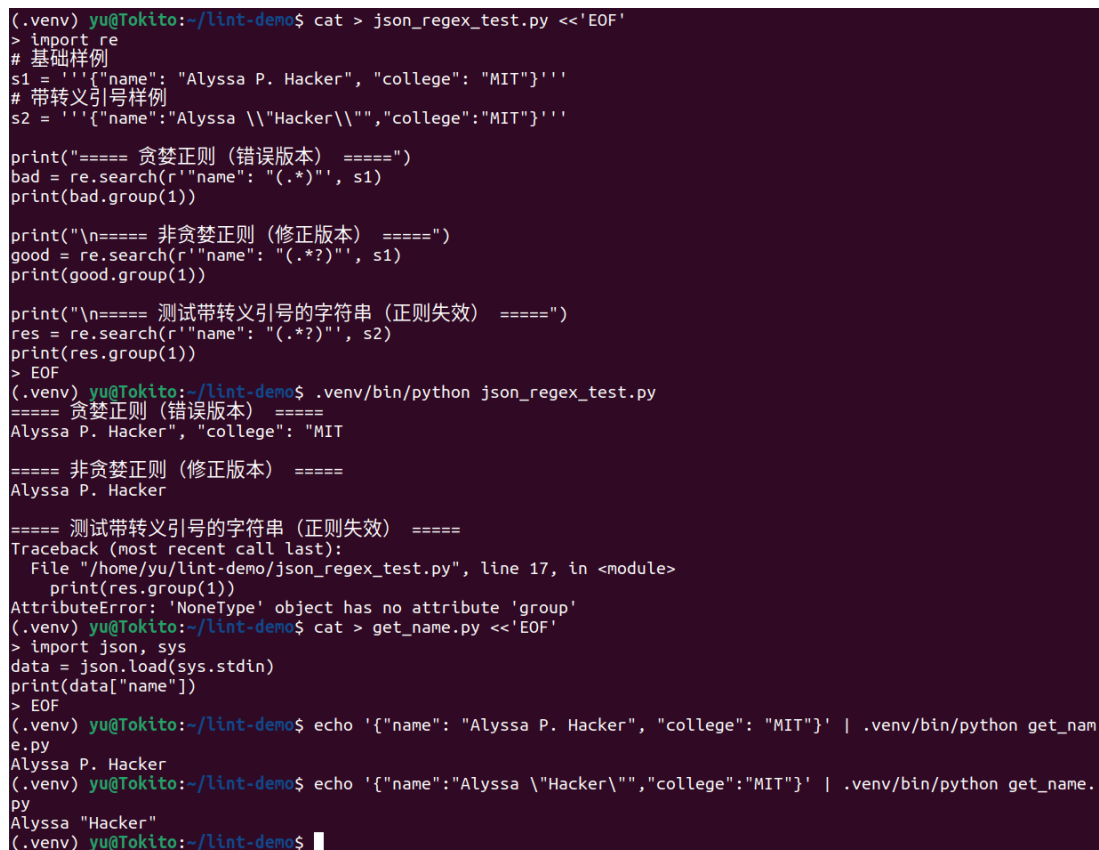
```
1 pattern = r'"name"\s*:\s*"((?:\\.|[^\"])*)"'
```

即便如此，正则仍需要自行处理 JSON 字符串、Unicode、转义和嵌套结构，正确做法是使用标准 JSON 解析器：

```
1 import json
2 import sys
3
4 data = json.load(sys.stdin)
5 print(data["name"])
```

命令行中可以简化为：

```
1 python -c 'import json,sys; print(json.load(sys.stdin)["name"])'
```



```
(.venv) yu@Tokito:~/lint-demo$ cat > json_regex_test.py <<'EOF'
> import re
# 基础样例
s1 = '{"name": "Alyssa P. Hacker", "college": "MIT"}'
# 带转义引号样例
s2 = '{"name": "Alyssa \\"Hacker\\"", "college": "MIT"}'

print("==== 贪婪正则（错误版本） =====")
bad = re.search(r'"name": "(.*)"', s1)
print(bad.group(1))

print("\n==== 非贪婪正则（修正版本） =====")
good = re.search(r'"name": "(.*)"', s1)
print(good.group(1))

print("\n==== 测试带转义引号的字符串（正则失效） =====")
res = re.search(r'"name": "(.*)"', s2)
print(res.group(1))
> EOF
(.venv) yu@Tokito:~/lint-demo$ .venv/bin/python json_regex_test.py
==== 贪婪正则（错误版本） =====
Alyssa P. Hacker, "college": "MIT"

==== 非贪婪正则（修正版本） =====
Alyssa P. Hacker

==== 测试带转义引号的字符串（正则失效） =====
Traceback (most recent call last):
  File "/home/yu/lint-demo/json_regex_test.py", line 17, in <module>
    print(res.group(1))
AttributeError: 'NoneType' object has no attribute 'group'
(.venv) yu@Tokito:~/lint-demo$ cat > get_name.py <<'EOF'
> import json, sys
data = json.load(sys.stdin)
print(data["name"])
> EOF
(.venv) yu@Tokito:~/lint-demo$ echo '{"name": "Alyssa P. Hacker", "college": "MIT"}' | .venv/bin/python get_name.py
Alyssa P. Hacker
(.venv) yu@Tokito:~/lint-demo$ echo '{"name": "Alyssa \\"Hacker\\"", "college": "MIT"}' | .venv/bin/python get_name.py
Alyssa "Hacker"
(.venv) yu@Tokito:~/lint-demo$
```

图 28: 课后练习六：对比贪婪与非贪婪正则并验证 JSON 解析器方案

对于两个输入样例，解析器分别输出 Alyssa P. Hacker 和 Alyssa "Hacker"，不受字段顺序、空格和转义规则影响。这说明结构化数据应优先交给结构化解析器，正则更适合约束清晰、格式简单的文本片段。

11 Git 仓库与提交记录

本周课堂实验按照 Q13、Q14、Q15 和 Q16 分目录组织。Q13 的提交记录了质量配置、测试用例和检查脚本三个阶段；Q14 使用 1f95941 提交增量构建流水线；Q16 则包含初始版本、故障修复以及 Makefile 和哈希文件的聚焦提交。相关证据分别见 5、8、19 和 20。

表 3: 本周主要 Git 提交记录

实验	提交编号	内容
Q13	07cc159	添加 pytest 与 Ruff 配置
Q13	ecbf6c4	添加正常与空白姓名测试
Q13	d2bc8d3	添加质量检查脚本
Q14	1f95941	添加 Make 构建流水线
Q16	d76a88b	初始化 Q16 项目
Q16	9da5a0c	添加检查与构建流程

12 实验总结

第四周实验把代码质量、构建系统和交付流程连接成一个完整闭环。Q13 通过 Ruff、pytest 和 `check.sh` 建立本地质量门禁；Q14 用 Makefile 显式描述源文件、中间文件和最终产物之间的依赖，验证增量构建与清理规则；Q15 使用 HTTP、curl 和 jq 把 API 数据转换为结构化 Markdown；Q16 在失败测试、修复、构建、哈希和 Git 提交之间完成一次可审计交付。

课后练习进一步补充了 pre-commit、覆盖率和持续集成。格式化器负责消除无关格式差异，linter 负责发现静态问题，测试负责验证行为，覆盖率报告负责指出未执行语句，CI 则把这些检查绑定到每次 push。AI agent 可以快速完成“运行工具、读取错误、修改代码、再次运行”的迭代循环，但必须向它提供真实工具结果和明确约束，并由人检查 diff。AI 生成测试也可能只追求覆盖率数字，因此要确认证言是否验证了真实业务行为。

正则部分展示了文本工具的边界。grep 适合快速搜索已知文本模式，semgrep 的 AST 匹配更能抵御换行和格式变化，但工具未安装时不能凭预期代替真实结果。Markdown 项目符号替换说明正则必须同时约束位置和上下文；JSON 姓名提取则更清楚地表明，复杂结构化数据应使用 JSON 解析器而不是不断堆叠正则。

综合来看，本周最重要的工程习惯是“所有结论都要有命令、输出和退出码支撑”。代码能运行、测试通过、构建成功和产物可追溯分别代表不同层次的验证，不能相互替代。以后在科研和项目开发中，我会优先把质量检查、测试、构建和 CI 固定为可重复命令，再借助 AI 加速迭代，同时保留人工审查和最终验证环节。